# Vetores

# Introdução

- Até agora vimos variáveis de tipos de dados básicos capazes de armazenar um dado de cada vez

- Existem casos onde é necessário armazenar uma grande quantidade de dados ao mesmo tempo
  - Entretanto, a criação de uma quantidade grande de variáveis é inviável Como resolver?
  - Solução: **vetores**!

- Um **vetor** (também chamado de **array**) é uma estrutura de dados composta por uma quantidade determinada de elementos de um mesmo tipo primitivo (inteiro, real, caractere, lógico)

- Um **vetor** é um agrupamento de dados de mesmo tipo representado por uma única variável

# Introdução

- Por ter como característica armazenar dados que seguem um mesmo tipo básico, diz-se que os vetores são **estruturas de dados homogêneas**

- Sabe-se que a variável funciona como um espaço na memória do computador, capaz de armazenar apenas um único dado

- Analogamente, um **vetor** pode ser considerado um conjunto de variáveis agrupadas (que ocupam vários de espaços na memória) referenciadas por uma única variável

- Mas como isso é possível?

# Vetores em C

- **Declaração de vetores em C:**

```
tipo nome[tamanho];
```

  - **tipo:** indica o tipo de cada elemento do vetor
  - **nome:** é o identificador da variável
  - **tamanho:** indica o tamanho do vetor, onde o menor valor é 1

- Exemplos:

```
int peso[10];
float nota[41];
char nome[80];
```

# Vetores em C

- **Atribuição de valores às posições do vetor:**
  - Como os vetores possuem várias posições, nas atribuições é necessário informar em qual delas o valor será armazenado
  - A posição inicia pelo índice zero
  - Exemplo: `float peso[5];`
    - **peso[0] = -4;** (atribui o valor -4 à primeira posição do vetor)
    - **peso[1] = 4;** (atribui o valor 4 à segunda posição do vetor)
    - **peso[3] = 90;** (atribui o valor 90 à quarta posição do vetor)
  - **Cuidado!**
    - **peso[5] = 15;** Tenta atribuir o valor 15 ao sexto elemento do vetor, mas o vetor só possui 5 posições

# Vetores em C

- **Preenchendo um vetor:**
  - Significa atribuir valores a todas as posições
  - Necessário utilizar algum mecanismo que controle o valor do índice

```
float  peso[5];
for (int i = 0 ;  i < 5 ;  i++) {
     printf("Digite o valor da posicao %d:", i);
     scanf("%f", &peso[i]);
}
```

  - Utilizou-se a estrutura de repetição **`for`**
  - Assim é possível garantir que variável **i** assuma todos os possíveis valores para o índice do vetor
  - Em cada execução do laço de repetição, uma posição do vetor será preenchida
  - Lembrar que o vetor inicia da posição zero!

# Vetores em C

- **Acessando valores do vetor:**
  - Para acessar os elementos do vetor, deve ser informado o índice para o qual se deseja obter o valor, juntamente com o nome da variável
  - Exemplo: `float peso [5];`
    - **peso[0]:** primeiro elemento do vetor
    - **peso[1]:** segundo elemento do vetor
    - **peso[2]:** terceiro elemento do vetor
    - **peso[3]:** quarto elemento do vetor
    - **peso[4]:** quinto elemento do vetor
  - **Cuidado!**
    - **peso[5]:** tenta acessar o sexto elemento do vetor, mas o vetor só possui 5 posições

# Vetores em C

- **Imprimindo os valores de um vetor:**
  - Significa exibir os valores das posições do vetor
  - Necessário utilizar algum mecanismo que controle o valor do índice
  - Utilizando o mesmo exemplo do vetor **float peso[5]**, o código abaixo exibe os valores de cada posição do vetor

```
for ( int i = 0 ; i < 5 ; i++ ) {
      printf("\n O valor da posicao %d e %f", i, peso[i]);
}
```

# Vetores em C

- **Inicialização do vetor no momento da sua declaração:**

```
int vet[5] = {10, 20, 30, 40, 50};     //  [10, 20, 30, 40, 50]
int vet[5] = {10, 20};             //  [10, 20, 0, 0, 0]
int vet[5] = { };                  //  [0, 0, 0, 0, 0]
int vet[5] = {10, 20, 30, 40, 50, 60}; //  [10, 20, 30, 40, 50]
Alerta!
int vet[5] = {[2] = 30};               //  [0, 0, 30, 0, 0]
int vet[ ] = {10, 20, 30};         //  [10, 20, 30]
```

# Exemplo – C

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  float notas[5], soma = 0, media = 0;
  for (int i = 0; i < 5; i++) {
    printf("Digite a nota %d: ", i + 1); // i+1 porque começa de zero
    scanf("%f", & notas[i]);
    soma = soma + notas[i];
  }
  media = soma / 5;
  printf("A media das notas e %.1f", media);
}
```

# Exercícios

1. Elabore um programa em C que gere 5 números inteiros aleatórios de 0 a 100 e ao final apresente cada um deles

# Exemplo – C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
  int i, tam = 5;
  srand(time(NULL));
  int vet[tam];
  printf("rand() = %d\n", rand());
  for (i = 0; i < tam; i++)
    vet[i] = rand() % 100;
  for (i = 0; i < tam; i++)
    printf("%d ", vet[i]);
  return 0;
}
```

# Exercícios

2. Elabore um programa em C que gere 5 notas aleatórias de 0 a 10 e ao final apresente as notas e a média

# Exemplo – C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
  int i, tam = 5;
  float vet[tam], soma;
  srand(time(NULL));
  printf("RAND_MAX = %d\n", RAND_MAX);
  printf("rand() = %d\n", rand());
  for (i = 0; i < tam; i++){
    vet[i] = ((float)rand()/RAND_MAX) * 10;
    soma += vet[i];
  }
  for (i = 0; i < tam; i++){
    printf("%f ", vet[i]);
  }
  printf("\nMedia: %.2f ", soma/tam);
  return 0;
}
```

# Exercícios

3. Elabore um programa em C que gere 10 números aleatórios de 0 a 100 e ao final apresente:

a) Quantos são pares
b) Quantos são ímpares
c) O maior número
d) O menor número

# Exercícios

4.Crie um programa que:

Leia 5 números inteiros e armazene em um vetor;
Exiba:

a) Todos os números digitados;
b) O maior valor do vetor.

# Exercícios

5. Crie um programa que:

a) Leia 10 números inteiros em um vetor;
b) Ordene os valores em ordem decrescente (sem usar funções prontas);
c) Remova valores repetidos (sem usar outro vetor);
d) Exiba o vetor final sem repetições.

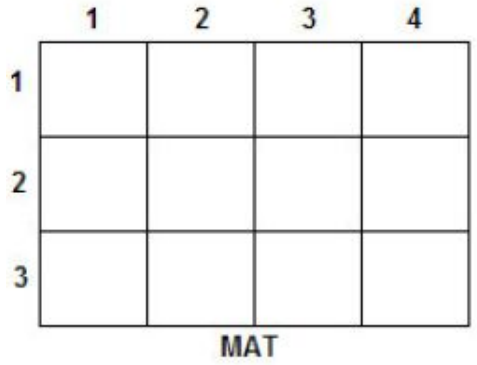
# Matrizes

# Introdução

- Vetores são variáveis **compostas unidimensionais**, isto é, **arranjos unidimensionais**
- Porém existem situações em que a natureza dos dados nos indica que forma de armazenamento possui mais de uma dimensão
  - Nesses casos, utilizamos **variáveis compostas multidimensionais**
  - De forma semelhante aos vetores, matrizes são identificadas por um nome e um tipo
  - Os elementos na matriz são diferenciadas pela especificação da posição dentro da estrutura

# Introdução

- Uma **matriz** é uma variável composta bidimensional
- Assim como um vetor, uma **matriz** armazena dados que seguem um mesmo tipo básico, sendo assim chamada de **estrutura de dados homogênea** (**bidimensional**)
- A variável referente a uma **matriz** atua como uma grade de linhas e colunas, onde a interseção entre uma linha e uma coluna armazena um dado
  - Exemplo: `MAT[3][4]`



# Matrizes em C

- Declaração de arranjos multidimesionais em C:
  - **tipo  nome[dimensão1] [dimensão2] ... [dimensão n]**
    - **tipo**: é o tipo de dados que a matriz pode armazenar
    - **nome**: é o identificador da matriz (nome dado à mesma)
    - **[dimensão1][dimensão2]...[dimensão n]**: representam as possíveis dimensões da matriz
  - Exemplos:
    - `int mat[4][10]`

      matriz **mat** de **inteiros** que possui **4 linhas** e **10 colunas**
    - `char mat[4][5]`

      matriz **mat** de **caracteres** que possui **4 linhas** e **5 colunas**

# Matrizes em C

- Preenchendo uma matriz:

```
int  mat[4][10];
for ( int i = 0 ; i < 4 ; i++ ) { //linhas
   for ( int j = 0 ; j < 10 ; j++ ) { //colunas
         printf("Digite o valor da linha %d coluna %d:", i, j);
         scanf("%d", &mat[i][j]);
   }
}
```

- Obs.: Uso de duas estruturas de repetição para acessar todos os valores da matriz.
  - O **for** externo (índice **i**) varia de 0 a 3 (percorrendo as 4 linhas)
  - O **for** interno (índice **j**) varia de 0 a 9 (percorrendo as 10 colunas)

# Matrizes em C

- Apresentando valores da matriz:

```
for ( int i = 0 ; i < 4 ; i++ ) { //linhas
    for ( int j = 0 ; j < 10 ; j++ ) { //colunas
      printf("%d ", mat[i][j]);
    }
    printf("\n");
}
```

- Obs.: Uso de duas estruturas de repetição para acessar todos os valores da matriz.
  - O **for** externo (índice **i**) varia de 0 a 3 (percorrendo as 4 linhas)
  - O **for** interno (índice **j**) varia de 0 a 9 (percorrendo as 10 colunas)

# Matrizes em C

- Usa-se lista de chaves para inicializar a matriz no momento da sua declaração

```
int mat[2][3] = { {10, 20, 30}, {40, 50, 60} };
 //  [   10, 20, 30,
 //  40, 50, 60  ]

int mat[2][3] = { {10, 20}, {40} };
 //  [   10, 20,  0,
 //  40,  0,  0  ]

int mat[2][3] = { {10}, { } };
 //  [   10, 0, 0,
 //   0, 0, 0    ]

int mat[2][3] = { };
 //  [   0, 0, 0,
 //  0, 0, 0 ]
```

# Matrizes em C

- Usa-se lista de chaves para inicializar a matriz no momento da sua declaração

```
int mat[2][3] = { {10, 20}, {[1]=40} };
 //  [   10, 20,  0,
 //   0, 40,  0  ]

int mat[2][3] = { {10, 20}, {[1]=40}, {50} }; // Alerta!
 //  [   10, 20,  0,
 //   0, 40,  0  ]

int mat[2][3] = { [0][1] = 20, [1][2]=30 };
 //  [     0, 20,   0,
 //    0,  0,  30    ]

int mat[2][3] = { 10, 20, 30, 40, 50, 60 };
 //  [     10, 20, 30,
 //    40, 50, 60    ]
```

# Matrizes em C

- Usa-se lista de chaves para inicializar a matriz no momento da sua declaração

```
int mat[ ][ ] = { {10, 20}, {30, 40} }; // Erro!

int mat[ ][3] = { {10, 20}, {30, 40} };
 //  [   10, 20, 0,
 //  30, 40, 0   ]

int mat[2][3] = { {10, 20}, {[1]=40, 50} };
 //  [   10, 20,  0,
 //   0, 40, 50  ]
```

# Arranjos Multidimensionais em C

- Exemplo para 3 dimensões

```
int  mat[4][5][10];
for ( int i = 0; i < 4; i++ ) { //dimensão 1
  for ( int j = 0; j < 5; j++ ) { //dimensão 2
     for ( int k = 0; k < 10; k++ ) { //dimensão 3
        printf("Digite o valor da dimensão %d, %d, %d", i, j, k);
        scanf("%d", &mat[i][j][k]);
       }
   }
}
```

# Observações

# Constantes em C

```
const int nLin = 4;
const int nCol = 3;
int  m[nLin][nCol];

for ( int i = 0 ; i < nLin ; i++ ) {
  for ( int j = 0 ; j < nCol ; j++ ) {
     printf("%d ", m[i][j]);
  }
   printf("\n");
}
```

- nLin e nCol são constantes, isto é, não podemos atribuir novos valores a essas variáveis
- Note que nLin e nCol foram usadas para declarar a matriz e também para as estruturas de repetição. Boa prática!

# Constantes em C

```
#define nLin 4
#define nCol 3

int  m[nLin][nCol];
for ( int i = 0 ; i < nLin ; i++ ) {
  for ( int j = 0 ; j < nCol ; j++ ) {
     printf("%d ", m[i][j]);
  }
  printf("\n");
}
```

- nLin e nCol são constantes, mas em nível de pré-processador!
- nLin e nCol não possuem tipo
- Se usarmos const, são variáveis e possuem tipo
- Se usarmos define, há "apenas" substituição de texto

# Arranjo de tamanho variável

- **Arranjos de tamanho variável (*variable-length array* - VLA)** são arranjos cujos tamanhos são determinados em tempo de execução ao invés de tempo de compilação
- Suponha que não se saiba, em tempo de compilação, qual o tamanho do arranjo e que essa informação será fornecida, por exemplo, pelo usuário no momento da execução do programa
- Nesse caso, precisamos ser capazes de declarar o array com tamanho definido no momento da execução do programa, isto é, com tamanho variável
- Para isso, usamos VLA

# Arranjo de tamanho variável

```
#include <stdio.h>

int main()
{
    int n;
    printf("Digite o tamanho do vetor: ");
    scanf("%d", &n);
    int v[n];
    for(int i=0; i<n; i++) {
        printf("Digite um numero: ");
        scanf("%d", &v[i]);
    }
    printf("Imprimindo o vetor: \n");
    for(int i=0; i<n; i++) {
        printf("%d ", v[i]);
    }
}
```

# Arranjo de tamanho variável

```
#include <stdio.h>

int main()
{
    int lin, col;
    printf("Digite a quantidade de linhas e colunas da matriz M: ");
    scanf("%d %d", &lin, &col);
    int mat[lin][col];
    printf("Preenchendo a matriz:\n");
    for(int i=0; i<lin; i++) {
        for(int j=0; j < col; j++){
            printf("M(%d,%d)=", i+1, j+1);
            scanf("%d", &mat[i][j]);
        }
    }
    printf("Imprimindo a matriz:\n");
    for(int i=0; i<lin; i++) {
        for(int j=0; j < col; j++){
            printf("%5d", mat[i][j]);
        }
        printf("\n");
    }
}
```

# Erro Atribuição Valores Arrays

Ao tentar inicializar uma matriz da seguinte forma:

```
const int lin = 3;
const int col = 2;
int mat[lin][col]={8,7,6,5,9,10};
```

é apresentado: **`error: variable-sized object may not be initialized`**

O erro ocorre porque, em C, o padrão ANSI C não permite a inicialização de variáveis de tamanho variável (*VLA - Variable Length Array*) na declaração

Embora **lin** e **col** sejam declarados como **const**, eles não são considerados constantes no tempo de compilação, mas sim variáveis de tempo de execução

Portanto, o compilador trata **mat** como um **VLA**. Os VLAs não podem ser inicializados diretamente, apenas podem ser atribuídos valores posteriormente na execução do programa

# Erro Atribuição Valores Arrays

Para resolver o problema, uma solução é substituir **lin** e **col** por macros **#define** ou usar **const** de uma maneira que o compilador possa considerar como constantes de tempo de compilação:

```
#define LIN 3
#define COL 2

int mat[LIN][COL] = {8, 7, 6, 5, 9, 10};
```

Ou, alternativamente, usar **enum** para definir as dimensões:

```
enum { lin = 3, col = 2 };
int mat[lin][col] = {8, 7, 6, 5, 9, 10};
```

Ambas as abordagens permitem a inicialização da matriz como você deseja, pois LIN e COL são considerados constantes no tempo de compilação

# Exercícios

1. Observe o programa a seguir. Qual a saída do programa?

```
#include <stdio.h>

int main()
{
  int t, i, m[3][4];
  for (t = 0; t < 3; t++) {
    for (i = 0; i < 4; i++) {
      m[t][i] = (t * 4) + i + 1;
    }
  }
  for (t = 0; t < 3; t++) {
    printf("\n");
    for (i = 0; i < 4; i++) {
      printf("%4d", m[t][i]);
      i < 3 ? printf("|") : 0;
    }
  }
  return 0;
}
```

# Exercícios

2. Faça um algoritmo que imprima a transposta de uma matriz

# Exercícios

```
#include <stdio.h>

int main()
{
   //int mat[5][5]={1,1,1,1,1,2,2,2,2,2,3,3,3,3,3,4,4,4,4,4,5,5,5,5,5};
   int mat[5][5]={1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25};
   int lin = 5;
   int col = 5;

   printf("Matriz Original\n");
   for(int i=0; i<lin; i++){
      for(int j=0; j<col; j++){
         printf("%02d ", mat[i][j]);
      }
      printf("\n");
   }

   printf("\nMatriz Transposta\n");
   for(int i=0; i<col; i++){
      for(int j=0; j<lin; j++){
         printf("%02d ", mat[j][i]);
      }
      printf("\n");
   }

   printf("\nDiagonal Principal da Matriz Transposta\n");
   for(int i=0; i<col; i++){
      for(int j=0; j<lin; j++){
         if(i == j)
            printf("%02d ", mat[j][i]);
      }
   }

   printf("\nDiagonal Secundaria da Matriz Transposta\n");
```

# Exercícios

3. Faça um algoritmo que imprima os elementos da diagonal principal de uma matriz quadrada

# Exercícios

4. Faça um algoritmo que imprima os elementos da diagonal secundária de uma matriz quadrada

# Exercícios

5. Faça um algoritmo que imprima os elementos abaixo da diagonal principal de uma matriz

# Exercícios

```
#include <stdio.h>

#define lin 3
#define col 3

int main() {
  int mat[lin][col] = {1,2,3,4,5,6,7,8,9};

  printf("Matriz Original\n");
  for (int i = 0; i < lin; i++) {
    for (int j = 0; j < col; j++) {
      printf("%02d ", mat[i][j]);
    }
    printf("\n");
  }

  printf("\nMatriz Transposta\n");
  for (int i = 0; i < col; i++) {
    for (int j = 0; j < lin; j++) {
      printf("%02d ", mat[j][i]);
    }
    printf("\n");
  }

  printf("\nDiagonal Principal da Matriz Transposta\n");
  for (int i = 0; i < col; i++) {
    for (int j = 0; j < lin; j++) {
      if (i == j)
        printf("%02d ", mat[j][i]);
    }
  }

  printf("\nDiagonal Secundaria da Matriz Transposta\n");
  for (int i = lin - 1; i >= 0; i--) {
    for (int j = 0; j < col; j++) {
      if (i + j == lin - 1)
        printf("%02d ", mat[j][i]);
    }
  }
  return 0;
}
```

# Exercícios

6. Faça um programa em C para preencher uma matriz com 5 notas aleatórias (0-10) de 10 alunos. Após isso, calcule, imprima e armazene em um vetor a média das notas de cada aluno e outro vetor com a média de cada avaliação

# Exercícios

```
#include <stdio.h>

#define lin 3
#define col 2

int main()
{
   float mat[lin][col]={8,7,6,5,9,10};
   float mediaAluno[lin];
   float mediaProva[col];
   float somaLinha = 0;
   float somaColuna = 0;

   printf("Matriz Original: \n");
   for(int i=0; i<lin; i++){
      for(int j=0; j<col; j++){
         printf("%04.1f ", mat[i][j]);
      }
      printf("\n");
   }

   for(int i=0; i<lin; i++){
      for(int j=0; j<col; j++){
         somaLinha+=mat[i][j];
      }
      mediaAluno[i] = somaLinha / col;
      somaLinha = 0;
   }

   for(int j=0; j<col; j++){
      for(int i=0; i<lin; i++){
         somaColuna+=mat[i][j];
      }
      mediaProva[j] = somaColuna / lin;
      somaColuna = 0;
   }

   printf("\nMedias Notas Alunos: \n");
   for(int i=0; i<lin; i++){
      printf("Media aluno %d: %.1f\n",i+1, mediaAluno[i]);
   }

   printf("\nMedias Notas Provas: \n");
```

# Exercícios

7. A distância percorrida em Km e a quantidade de litros de combustível utilizados por uma pequena frota de 8 taxis de uma cooperativa em um determinado período são apresentadas a seguir:

| **Taxi** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| **Distância (km)** | 1.233 | 999 | 1.566 | 1.348 | 1.088 | 875 | 1.576 | 1.321 |
| **Consumo Combustível (litros)** | 120 | 87 | 130 | 140 | 95 | 70 | 145 | 125 |

Tendo como base os dados acima, elabore um programa em C que execute as seguintes tarefas:

a) Calcular e apresentar a distância total percorrida pela frota
b) Calcular e apresentar o consumo total de combustível consumido pela frota
c) Calcular e apresentar o consumo médio em km/litro de cada taxi
d) Apresentar quais foram os taxis mais e menos econômicos da frota

# Exercícios

```
#include <stdio.h>

#define tamFrota 8

int main() {
  int distancias[tamFrota] = {1233,999,1566,1348,1088,875,1576,1321};
  int consumos[tamFrota] = {120,87,130,140,95,70,145,125};
  float consumosMedios[tamFrota];
  int i, somaDist, distTotal, somaConsumo, consumoTotal;
  float maior, menor;
  int posMaior, posMenor;

  somaDist = 0;
  somaConsumo = 0;

  for (i = 0; i < tamFrota; i++) {
    somaDist += distancias[i];
    somaConsumo += consumos[i];
    consumosMedios[i] = (float) distancias[i] / consumos[i];
  }

  maior = 0;
  menor = 999;
  printf("Consumos Medios Frota (km/litro): ");
  for (i = 0; i < tamFrota; i++) {
    printf("%.2f ", consumosMedios[i]);
    if (consumosMedios[i] >= maior) {
      maior = consumosMedios[i];
      posMaior = i;
    }

    if (consumosMedios[i] < menor) {
      menor = consumosMedios[i];
      posMenor = i;
    }
  }

  printf("\nDistancia total percorrida pela frota: %d km\n", somaDist);
  printf("Total de combusivel consumido pela frota: %d litros\n", somaConsumo);
  printf("Consumo medio da frota: %.2f km/litro\n", (float) somaDist / somaConsumo);
  printf("Maior consumo: Taxi %d \n", posMaior + 1);
  printf("Menor consumo: Taxi %d \n", posMenor + 1);

  return 0;
```

# Referências bibliográficas


- ASCENCIO, A. F. G. e CAMPOS, E. A. V. **Fundamentos da Programação de Computadores – Algoritmos, Pascal e C/C++. São Paulo: Pearson Prentice Hall, 2007. 2ª Edição.**
- SOUZA, A. Furlan; GOMES, Marcelo Marques; SOARES, Marcio Vieira e CONCILIO, Ricardo. **Algoritmos e Lógica de Programação. 2ª ed. Ver. e ampl. São Paulo: Cengage Learning 2011.**

# Outros Exemplos

# Referências

Guia definitivo para gerar valores aleatórios em C
https://www.galirows.com.br/meublog/programacao/guia-definitivo-valores-aleatorios-c/

```
// função para gerar e testar números decimais aleatórios
// utilizando faixa de valores

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

float aleatorio(int min, int max){
    return ((rand()/(float)RAND_MAX) * (max - min) + min);
}

int main(void) {
    srand(time(NULL));
    float num;
    int inicio = 10;
    int fim = 40;
    int numAbaixo = 0;
    int numAcima = 0;
    int limiteInferior = 0;
    int limiteSuperior = 0;
    int totalNumeros = 0;
    for (int i=0; i<1000; i++){
        num = aleatorio(inicio,fim);
        printf("%.1f ", num);
        if (num < inicio) numAbaixo++;
        if (num > fim) numAcima++;
        if (num == inicio) limiteInferior++;
        if (num == fim) limiteSuperior++;
        if (num >= inicio || num <= fim) totalNumeros++;
}
printf("\nnumAbaixo=%d - numAcima=%d\n", numAbaixo, numAcima);
printf("limiteInferior=%d - limiteSuperior=%d\n", limiteInferior, limiteSuperior);
printf("totalNumeros=%d\n", totalNumeros);
return 0;
}
```

# Preenchendo e Apresentando uma Matriz de uma faixa de números decimais

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

float aleatorio(int min, int max){
   return ((rand()/(float)RAND_MAX) * (max - min) + min);
}

int main(void) {
   srand(time(NULL));
   int linhas = 4;
   int colunas = 5;
   float temperaturas[linhas][colunas];

   for(int i=0; i<linhas; i++){
      for(int j=0; j<colunas; j++){
         temperaturas[i][j]=aleatorio(10,40);
      }
   }
   for(int i=0; i<linhas; i++){
      for(int j=0; j<colunas; j++){
         printf("%.1f ",temperaturas[i][j]);
      }
      printf("\n");
   }
return 0;
}
```

# Manipulando matrizes por referência

https://www.ime.usp.br/~leo/mac2166/2017-1/line_introducao_matrizes.html

```
#include <stdio.h>
#define NL 20 // constante para maximo de linhas
#define NC 20 // constante para maximo de colunas
// Funcao para gerar nova matriz MA + MB
void somaMat (int MA[][NC], int MB[][NC], int MC[][NC], int nL, int nC) {
  int i, j; // auxiliar, para indexar os matrizes
  for (i=0; i< nL; i++)   // "Ler" nL x nC valores
    for (j=0; j< nC; j++) // inteiros armazenando-os
      MC[i][j] = MA[i][j] + MB[i][j];
  }
// Funcao para entrar matriz nL x nC, linha por linha
void lerMatriz (int mat[][NC], int nL, int nC) {
  int i,j; // declara indices
  for (i=0; i<nL; i++)   // i entre 0 e nL-1
    for (j=0; j<nC; j++) // j entre 0 e nC-1
      scanf("%d", &mat[i][j]);
  }
// Funcao para imprimir matriz nL x nC, linha por linha
void imprimeMatriz (int mat[][NC], int nL, int nC) {
  int i,j; // declara indices
  for (i=0; i< nL; i++) { // i entre 0 e nL-1
    printf("%2d: ", i); // "Imprime" numero desta linha
    for (j=0; j< nC; j++) // j entre 0 e nC-1
      printf("%3d ", mat[i][j]); // em 3 espacos
    printf("\n"); // Mude de linha
    }
  }
int main (void) {
  int i, j; // auxiliar, para indexar os matrizes
  int m, n; // tamanho util das matrizes
  int matA[NL][NC], matB[NL][NC], matC[NL][NC]; // Matriz para inteiros (ate' NL*NC elementos)
  scanf("%d %d", &m, &n);
```

# Leitura e escrita de uma matriz por referência da até 20 colunas

```
#include <stdio.h>

void lerMatriz (int mat[][20], int nL, int nC) {
  int i,j;
  for (i=0; i<nL; i++)
    for (j=0; j<nC; j++)
      scanf("%d", &mat[i][j]);
  }
// Funcao para imprimir matriz nL x nC, linha por linha
void imprimeMatriz (int mat[][20], int nL, int nC) {
  int i,j;
  for (i=0; i< nL; i++) {
    printf("%2d: ", i);
    for (j=0; j< nC; j++)
      printf("%3d ", mat[i][j]);
    printf("\n");
    }
  }

int main (void) {
  int i, j; // auxiliar, para indexar os matrizes
  int m, n; // tamanho util das matrizes
  int matA[m][n]; // Matriz para inteiros (ate' NL*NC elementos)
  printf("Entre com as dimensoes da matriz (linha coluna): \n");
  scanf("%d %d", &m, &n);
  printf("Entra matriz A:\n"); lerMatriz(matA, m, n);
  printf("Matriz A:\n"); imprimeMatriz(matA, m, n); // 'matA' e' parametro efetivo

  return 0;
  }
```

# Leitura e escrita de uma matriz – outro exemplo

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_COL 100
#define MAX_ROW 100


float aleatorio(int min, int max){
   return ((rand()/(float)RAND_MAX) * (max - min) + min);
}

void preencherMatriz(int matriz[][MAX_COL], int linhas, int colunas) {
   char op;
   printf("Deseja preencher manualmente (s/n)?: ");
   scanf(" %c",&op);
   for (int i = 0; i < linhas; i++) {
      for (int j = 0; j < colunas; j++) {
         if(op=='s'){
            printf("Digite o valor para a posição [%d][%d]: ", i, j);
            scanf("%d", &matriz[i][j]);
         } else
            matriz[i][j] = aleatorio(0,100);
      }
   }
}

void imprimeMatriz(int matriz[][MAX_COL], int linhas, int colunas) {
   for (int i = 0; i < linhas; i++) {
      for (int j = 0; j < colunas; j++) {
         printf("%02d ", matriz[i][j]);;
      }
```

# Vetor de Strings

```
#include <stdio.h>

int main()
{
   char meses[12][4] = {"Jan","Fev","Mar","Abr","Mai","Jun","Jul","Ago","Set","Out","Nov","Dez"};
   int numMes;
   printf("Entre com o numero do mes: ");
   scanf("%d",&numMes);
   printf("%s\n", meses[numMes-1]);
   printf("Todos os meses: \n");
   for(int i=0; i<12; i++)
      printf("%02d=%s\n", i+1, meses[i]);
   return 0;
}
```

# Multiplicação Matrizes Quadradas

```
#include <stdio.h>
#define NL 3
#define NC 3

int main () {

  int A[NL][NC] = { 1, 1, 1, 2, 2, 2, 3, 3, 3 };
  int B[NL][NC] = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
  int C[NL][NC];
  int i, j, k, soma;

  for (i = 0; i < NL; i++){
     for (j = 0; j < NC; j++){
        soma = 0;
        for (k = 0; k < NL; k++){
           soma += A[i][k] * B[k][j];
           C[i][j] = soma;
        }
     }
  }
  for (i = 0; i < NL; i++){
     for (j = 0; j < NL; j++){
        printf ("%02d ", C[i][j]);
     }
     printf ("\n");
  }
  return 0;
}

// 12 15 18
// 24 30 36
// 36 45 54

// https://www.somatematica.com.br/emedio/matrizes/matrizes4.php
// https://homepages.dcc.ufmg.br/~rodolfo/aedsi-2-10/Arranjo/multiplica-matrizes.html
```